

Chat-IA — Proceso de QA

Chat-IA

July 31, 2026

CONTENTS

Proceso de QA — Chat-IA	
1. Alcance ejecutado	
2. Cómo se ejecutó (orquestración con agentes)	
3. Resultado de Jest (pruebas unitarias)	
4. Resultado de Playwright (pruebas e2e, Chrome)	
5. Bug real encontrado y corregido	
6. Auditoría de dependencias	
7. Auditoría de valores hardcodeados	
8. Base de datos	
9. Revisión de usabilidad	
10. Estado final	

Proceso de QA — Chat-IA

Documenta la ejecución real del plan de pruebas (`TEST_PLAN.md`): qué se corrió, con qué herramientas, qué falló, qué se corrigió y qué quedó pendiente. Generado el 2026-08-01.

1. Alcance ejecutado

A diferencia de `TEST_PLAN.md` (que es el plan, sin ejecutar), este documento cubre la **ejecución real**:

- Pruebas unitarias con **Jest** sobre funciones puras/aisladas.
- Pruebas end-to-end con **Playwright Test** (`@playwright/test`), navegador **Chromium únicamente**, contra la app real corriendo en `http://localhost:3000` con Postgres y Ollama reales (sin mocks).
- Auditoría de dependencias desactualizadas.
- Auditoría de valores hardcodeados.
- Revisión de índices de la base de datos.
- Un bug real encontrado por los tests fue corregido en el propio código de la app.

2. Cómo se ejecutó (orquestración con agentes)

Dado el volumen de trabajo, se usaron dos agentes en paralelo para el setup inicial (sin superposición de archivos entre ellos):

- **Agente A** — instaló y configuró Jest (`ts-jest` , `jest.config.js`), exportó funciones antes privadas para poder testearlas (`parseAttachments` en `src/app/api/chat/route.ts` , `parseNdjsonLine` extraída de `src/lib/ollama.ts`), movió `shortModel` / `isCloudModel` a `src/lib/model-utils.ts` , y escribió 30 tests.
- **Agente B** — instaló `@playwright/test` , generó fixtures propias (sin datos externos), y escribió 24 tests e2e cubriendo las categorías F-CHAT, F-MODEL, F-ATTACH, F-DEL, F-SETTINGS, F-MEMORY, F-RESP y validación de formularios del `TEST_PLAN.md` .

Ambos agentes tocaron `package.json` (cada uno agregó su script `test` / `test:e2e`) sin conflicto real — el merge fue automático y limpio.

Después de que ambos terminaran, el proceso continuó de forma secuencial (no paralela) porque requería iterar contra el servidor real: correr las suites, diagnosticar fallas, corregir, y volver a correr.

3. Resultado de Jest (pruebas unitarias)

```
PASS src/lib/ollama.test.ts
PASS src/app/api/chat/route.test.ts
PASS src/lib/settings.test.ts
PASS src/lib/model-utils.test.ts

Test Suites: 4 passed, 4 total
Tests:       30 passed, 30 total
```

Cobertura: `buildOllamaMessage` (armado de mensajes multimodales), `parseNdjsonLine` (parser del streaming de Ollama), `getSystemPrompt` / `setSystemPrompt` (lectura/escritura de `SYSTEM_PROMPT.md`, con `node:fs/promises` mockeado), `parseAttachments` (validación de adjuntos), `shortModel` / `isCloudModel` (utilidades de nombre de modelo).

Comando: `npm test` (`jest`).

4. Resultado de Playwright (pruebas e2e, Chrome)

Corrida final, limpia, tras corregir el bug de la sección 5:

```
Running 24 tests using 1 worker
24 passed (1.0m)
```

Comando: `npm run test:e2e` (`playwright test --project=chromium`). Reporte HTML generado en `playwright-report/` (`npx playwright show-report`, sirve en `http://localhost:9323`).

Nota de estabilidad: el caso `F-CHAT-05` (doble click en Enviar) depende de que Ollama responda dentro de un timeout de 90s. En corridas posteriores se observó que falla en el primer intento por timeout de red hacia el modelo cloud (`gpt-oss:20b-cloud`, corre en los servidores de Ollama, no local) y pasa en el reintento — es latencia externa variable, no una regresión de la app. Confirmado corriendo el test en aislamiento con `--retries=2`: marca “flaky” (falla intento 1, pasa intento 2), consistente con causa externa. No se puede eliminar esta variabilidad sin cambiar de modelo a uno local real.

5. Bug real encontrado y corregido

F-CHAT-05 — doble click en “Enviar” creaba dos conversaciones duplicadas.

- **Causa raíz:** en `src/app/page.tsx`, función `handleSend`, el guard `setIsStreaming(true)` se ejecutaba **después** del `await fetch("/api/conversations", ...)` que crea la conversación cuando todavía no había `activeId`. Durante esa ventana asíncrona, el botón de enviar seguía habilitado (el input seguía teniendo texto e `isStreaming` seguía en `false`), así que un segundo click volvía a entrar a `handleSend`, encontraba `activeId` todavía `null`, y creaba una segunda conversación con el mismo primer mensaje.
- **Fix aplicado:** se movió `setIsStreaming(true)` a la primera línea de `handleSend`, antes de cualquier `await`, con reset a `false` si la creación de la conversación falla. Como el guard ahora se ejecuta de forma síncrona en el primer click, el segundo evento de click (que el navegador despacha después, de forma serializada) encuentra el botón ya deshabilitado.
- **Validación:** `F-CHAT-05` pasa en la corrida limpia (sección 4) y en la corrida de regresión general (`qa_intensive` con doble-click, ya usado antes en la sesión).

6. Auditoría de dependencias

Ver `TEST_PLAN.md` sección 4 para el detalle completo. Resumen: `next/eslint-config-next` ya al día (16.2.12); `react/react-dom` con un patch disponible (19.2.4 → 19.2.8, bajo riesgo); `typescript`, `prisma/@prisma/client` y `@types/node` tienen saltos de versión **mayor** disponibles (TS 5→7, Prisma 6→7) que **no se aplicaron** — requieren revisión de breaking changes antes de actualizar, fuera del alcance de esta ronda de QA.

7. Auditoría de valores hardcodedos

Se re-auditó todo `src/` después del rediseño y el rename a “Chat-IA”. Se encontraron y corrigieron dos textos de copy desactualizados (no afectaban funcionalidad, solo mensajes al usuario):

1. `src/app/api/chat/route.ts` — mensaje de error de fallback usaba el string literal `"localhost:11434"` en vez de `getBaseUrl()` real. Corregido en la sesión (antes de esta ronda de QA formal).
2. `src/app/page.tsx` — el placeholder del input cuando no hay modelos decía *“Configura OLLAMA_MODELS en .env”*, una variable que **ya no existe** (la lista de modelos se detecta en vivo desde Ollama, no desde `.env`, desde un cambio anterior en la misma sesión). Corregido a *“No hay modelos instalados en Ollama”*.

No quedan URLs, puertos, ni credenciales hardcodedas fuera de los defaults esperados (`getBaseUrl()` en `src/lib/ollama.ts` usa `http://localhost:11434` como fallback documentado, no como valor fijo — se pisa con `OLLAMA_BASE_URL`).

8. Base de datos

Se agregaron dos índices que no existían, alineados a los queries reales de la app (ver `prisma/schema.prisma`, migración `optimize_indexes`):

- `Conversation`: `@@index([updatedAt])` — soporta el `ORDER BY updatedAt desc` del listado del sidebar.
- `Message`: se reemplazó `@@index([conversationId])` por el compuesto `@@index([conversationId, createdAt])` — cubre el filtro por conversación y el `ORDER BY createdAt asc` sin un sort aparte.

A la escala actual (un solo usuario, cientos de filas como mucho) el impacto real es marginal, pero corrige la falta de cobertura de índice para los patrones de consulta reales de la app.

9. Revisión de usabilidad

Cubierta de forma indirecta por los propios tests funcionales (F-RESP para mobile, F-SETTINGS/F-MEMORY para el flujo de configuración) y por el rediseño visual aplicado en la misma sesión (paleta profesional, tipografía sans neutra, foco/teclado accesible en ambos modales). No se identificaron bloqueos de usabilidad adicionales más allá de lo ya recogido en `TEST_PLAN.md` sección “Notas de seguridad” y los hallazgos de la revisión de código (contexto sin límite en conversaciones largas, errores de `DELETE` silenciados).

10. Estado final

Verificación	Resultado
<code>npx tsc --noEmit</code>	✓ limpio
<code>npm test</code> (Jest)	✓ 30/30
<code>npm run test:e2e</code> (Playwright, Chrome)	✓ 24/24 (1 caso con flakiness externa documentada)
Auditoría de hardcodeo	✓ sin hallazgos pendientes
Bug real encontrado	✓ corregido y validado
Índices de DB	✓ optimizados para los queries reales